MANUAL DE BANCADA • 24/10/2026

EPROM 27C256
Teste, leitura e gravação

Arduino Mega 2560 + ST 27C256-30CQ

Guia completo com pinagem, ligação de leitura, diagnóstico, exportação de 32 KB, apagamento UV, circuito de referência para programação e códigos integrais para Arduino e computador.

Dois níveis de validação. A leitura de sua montagem foi demonstrada na conversa. O gravador descrito aqui é um projeto de referência com revisão técnica e software verificado; não foi montado nem ensaiado fisicamente. As fontes de programação são ajustadas manualmente, com multímetro.

O que já foi observado

Evidência	Resultado
Três leituras consecutivas	CRC32 = 6691559F em todas
Troca de endereço e retorno	0 divergências; máscara 0x00
Conteúdo lido	17.064 bytes FF; 1.378 bytes 00
Conclusão proporcional	Leitura estável; a memória não estava vazia

Esses resultados não identificam os jogos nem comprovam todas as células ou todas as linhas de endereço. A comparação com uma imagem conhecida e o ciclo apagar/gravar/verificar são verificações adicionais.

Como usar este manual

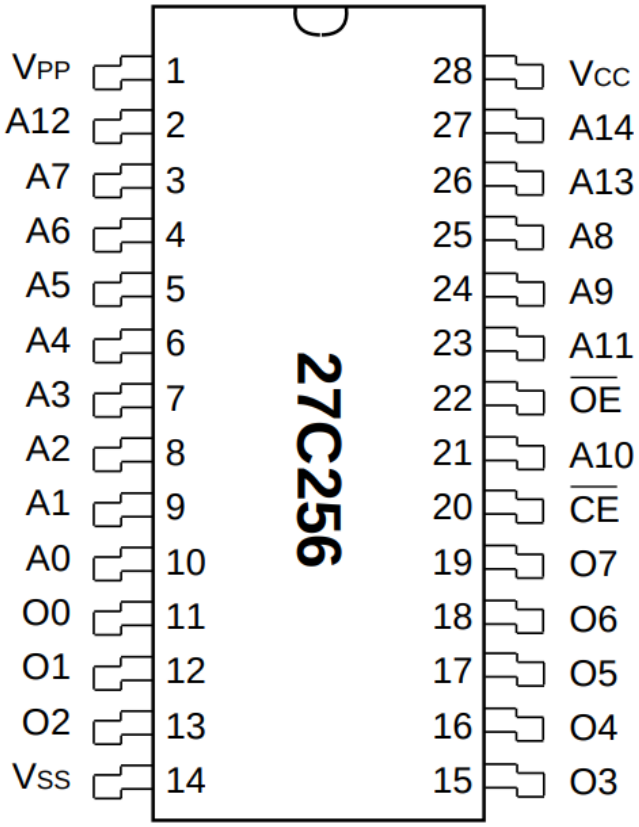
Comece pelas seções 1 a 5 para repetir a leitura. As seções 6 a 10 descrevem o gravador de bancada. Os apêndices trazem o firmware e o cliente Python completos. Os mesmos arquivos estão no pacote de códigos que acompanha o PDF.

Escopo: DIP-28, 32.768 × 8 bits. Os valores de programação são da **TS27C256Q antiga**, cujo datasheet lista TS27C256-30CQ. A imagem enviada mostra a pinagem, mas não confirma o texto completo gravado no seu chip. Confira essa identificação antes de programar; não transfira esses valores para M27C256B ou outros fabricantes.

1 • IDENTIFICAÇÃO

Pinagem da EPROM

Vista superior, com o entalhe para cima: pino 1 no canto superior esquerdo, descendo até 14; pelo lado direito, a numeração sobe de 15 até 28. A figura abaixo foi fornecida por você.



Sinal	Função
A0 a A14	15 entradas de endereço; 0x0000 a 0x7FFF
O0 a O7 / Q0 a Q7 / D0 a D7	Oito linhas de dados; nomes equivalentes neste manual
/CE e /OE	Ativos em nível baixo; barra superior significa inversão
VSS, VCC e VPP	Terra, alimentação e alimentação de programação

O “A0” da EPROM não é o conector analógico A0 do Mega. Aqui, A0 da EPROM é ligado ao pino digital D22 do Mega.

2 • LEITOR DIRETO DE 5 V

Ligação pino a pino

Esta é a **montagem A**: apenas leitura, com **PROGRAMMER_HARDWARE = 0**. A tabela padroniza o mapa final com D47 e conserva a troca entre Q0 e Q2 feita durante o diagnóstico. Refaça ou confira o mapa inteiro; não misture tabelas antigas da conversa.

Pino EPROM	Sinal	Mega / alimentação
1	VPP	5V
2	A12	D34
3	A7	D29
4	A6	D28
5	A5	D27
6	A4	D26
7	A3	D25
8	A2	D24
9	A1	D23
10	A0	D22
11	Q0	D47
12	Q1	D38
13	Q2	D37
14	VSS	GND
15	Q3	D40
16	Q4	D41
17	Q5	D42
18	Q6	D43
19	Q7	D44
20	/CE	D45
21	A10	D32
22	/OE	D46
23	A11	D33
24	A9	D31
25	A8	D30
26	A13	D35
27	A14	D36
28	VCC	5V

Controles: resistor de 10 kΩ de /CE para 5 V e outro de /OE para 5 V. **Desacoplamento:** 100 nF de 28 a 14 e outro de 1 a 14, com fios curtos. Retire a memória do cartucho antes de conectá-la ao leitor.

3 • MONTAGEM E ALIMENTAÇÃO

Antes de iniciar a leitura

Sequência para a montagem A

1. Com a EPROM desconectada, envie o firmware configurado como leitor. Escolha Arduino Mega or Mega 2560 na IDE e a porta correta.
2. Retire o USB. Monte e confira as 28 conexões.
3. Confirme que os pinos 1 e 28 recebem o mesmo 5 V e que 14 é GND.
4. Reconecte o USB. Abra a serial em 115200 baud ou execute o cliente Python. Feche o Monitor Serial antes de abrir a mesma porta pelo Python.

Componente	Aplicação
2 × 100 nF cerâmicos (104), 25 V ou mais	Um de VCC para GND e outro de VPP para GND, junto ao chip
2 × 10 kΩ	Pull-ups de /CE e /OE no leitor direto
Eletrolíticos de 100 µF	Filtragem adicional; não equivalem a 100 nF
Multímetro	Medir entre 28 e 14, depois entre 1 e 14

100 nF = 0,1 µF; 100 µF é mil vezes maior. Em eletrolíticos, positivo na alimentação e negativo no GND. Para leitura a 5 V, use tensão nominal de pelo menos 6,3 V; para VPP de programação, prefira 25 V ou mais. O capacitor do Mega não substitui o capacitor perto da EPROM.

Um ensaio inicial sem os cerâmicos pode funcionar, como ocorreu durante a conversa. Se houver variação, não conclua defeito no chip sem corrigir alimentação, contatos e desacoplamento. No gravador, os 100 nF fazem parte da montagem especificada.

O que não aplicar ao leitor direto

A montagem A fica exclusivamente em 5 V. Não aplique 6 V ou 12,5 V a ela. O circuito da montagem B altera as conexões de dados e controles; mudar somente o código ou a fonte não transforma o leitor em gravador.

4 • SOFTWARE NO COMPUTADOR

Ler os 32 KB e verificar vazio

Arquivos do pacote: **eprom27/eprom27.ino** e **eprom_pc.py**. O firmware fica aguardando comandos; esta versão não inicia automaticamente as três leituras ao ligar. Todas as operações são solicitadas pelo cliente ou pelo Monitor Serial.

Preparação do Python 3

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install pyserial
```

No Windows: crie o ambiente com **py -m venv .venv** e ative com **.venv\Scripts\activate** no Prompt de Comando. Troque /dev/ttyACM0 pela porta real, por exemplo COM3. Em Linux, o nome também pode ser /dev/ttyUSB0.

Operações de leitura

```
python eprom_pc.py --porta /dev/ttyACM0 info
python eprom_pc.py --porta /dev/ttyACM0 teste
python eprom_pc.py --porta /dev/ttyACM0 vazio
python eprom_pc.py --porta /dev/ttyACM0 ler original.bin
python eprom_pc.py --porta /dev/ttyACM0 verificar original.bin
```

ler faz três dumps completos, compara os 32.768 bytes entre si e só salva se forem idênticos. Não sobrescreve arquivo existente. **vazio** usa o mesmo teste e informa se todos os bytes são FF. **verificar** compara a memória com um arquivo de exatamente 32 KB.

Comandos pelo Monitor Serial

Enviar	Operação
I	Identificação do firmware e do tipo de montagem
T	Três CRCs/contagens e teste de troca de endereço
B	Uma varredura com CRC, contagens e BLANK/NOT_BLANK
D	Desarma e deixa endereços em alta impedância

Não envie R no Monitor Serial para visualizar texto: R transmite um arquivo binário. Use o cliente para recebê-lo. O comando D não desconecta fontes externas.

5 • DIAGNÓSTICO

Estabilidade, vazio e conteúdo correto

Resultado	Interpretação
CRCs iguais e DIFF=0	Repetibilidade demonstrada nos ensaios realizados
FF=32768 e ZERO=0	Compatível com memória vazia; descartar falha de habilitação antes de aceitar
FF menor que 32768	Existem bytes diferentes de FF, se a leitura é estável
Comparação exata com BIN conhecido	Conteúdo coincide nos 32.768 endereços lidos

“Vazia” significa apagada para todos os bits em 1. Não significa uma sequência de zeros. Um barramento desconectado ou saídas desabilitadas também pode aparentar FF; o blank check isolado não comprova integridade.

O defeito de conexão observado na conversa

A máscara 0x04 apontou divergência no bit Q2 (pino 13). Após trocar as pontas de Q0 e Q2 e ajustar o array, a máscara mudou para 0x01. A falha acompanhou o caminho de fio/entrada D39. Depois da correção, o CRC ficou em 6691559F e DIFF foi zero. Isso não demonstrou que o próprio pino D39 estava danificado: jumper e contatos pertenciam ao mesmo caminho suspeito.

Máscara	Bit	Pino EPROM	Mega atual
0x01	Q0	11	D47
0x02	Q1	12	D38
0x04	Q2	13	D37
0x08	Q3	15	D40
0x10	Q4	16	D41
0x20	Q5	17	D42
0x40	Q6	18	D43
0x80	Q7	19	D44

Se voltar a variar

Desligue antes de mexer. Revise o fio indicado pela máscara, GND, VPP e VCC, encaixe do chip e todos os endereços. Use fios curtos e os cerâmicos. Se houver multímetro, meça as tensões no próprio chip durante a varredura. Leituras corretas em um único endereço não eliminam falhas nos demais.

6 • APAGAMENTO E ARQUIVO

Preparar uma nova imagem

Você autorizou descartar os quatro jogos existentes. Ainda assim, o leitor permite salvar a imagem atual se quiser. A memória não é apagada pelo Mega: a versão com janela de quartzo usa apagamento UV. Se o encapsulamento não tiver janela, não presuma que possa reutilizá-lo após gravado.

Apagamento UV

Use apagador fechado próprio para EPROM, com UV-C de aproximadamente 254 nm. Retire o chip do circuito e exponha a janela conforme as instruções do equipamento. O tempo depende de intensidade, distância e envelhecimento da lâmpada; não use um tempo fixo como prova de apagamento. Não exponha olhos ou pele ao UV. Depois, recoloque no leitor e execute o comando **vazio**. Só aceite 32.768 bytes FF com leitura estável. Cubra a janela após finalizar o uso. [1]

Criar uma imagem de teste de 32 KB

Para o primeiro ciclo de programação, uma imagem pseudoaleatória conhecida ajuda a testar combinações de endereço e dados. O arquivo abaixo não é um jogo e não deve ser executado no Atari.

```
# Salve como criar_teste.py e execute: python criar_teste.py
import random
from pathlib import Path
r = random.Random(2600)
dados = bytes(r.randrange(256) for _ in range(32768))
Path('teste32k.bin').write_bytes(dados)
```

Esse código também está no pacote como criar_teste.py. Execute em uma pasta de trabalho; o arquivo teste32k.bin será criado ou substituído. O cliente não completa, duplica nem reorganiza arquivos automaticamente.

Quatro jogos em uma EPROM

A capacidade de 32 KB não determina sozinha onde os quatro jogos ficam. O layout depende de como o cartucho seleciona os endereços e de eventual bankswitching dos jogos. Identifique a placa e os tamanhos antes de concatenar ROMs. O gravador grava bytes na ordem do BIN; ele não implementa o circuito do cartucho.

7 • GRAVADOR DE REFERÊNCIA

Montagem B: componentes e arquitetura

Somente para a TS27C256Q antiga confirmada. Montagem de bancada ainda não ensaiada fisicamente. Configuração do firmware: PROGRAMMER_HARDWARE = 1. Os sinais do Mega em D45/D46 são gates de MOSFETs, e não conexões diretas aos pinos 20/22.

A interface usa oito MOSFETs para escrever dados, dois para /CE e /OE, e dois LM339 para receber os dados. Os comparadores têm saída de coletor aberto puxada para 5 V; assim, as entradas do Mega não recebem os até 6,25 V possíveis no lado da EPROM. Os endereços continuam diretos, pois os níveis de entrada da EPROM são TTL.

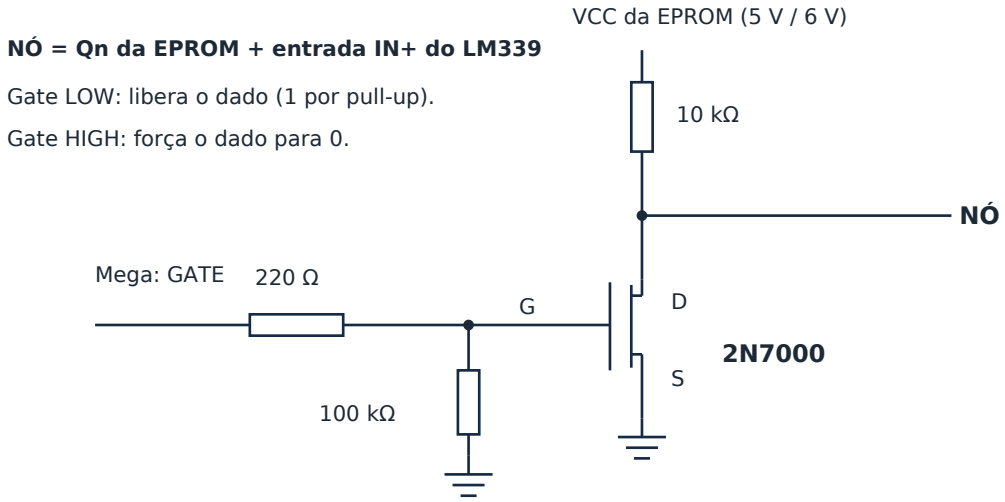
Quantidade	Componente / função
1	Mega 2560 e soquete DIP-28 de 600 mil, preferencialmente ZIF
2	LM339 em DIP-14; não confundir com LM393 de dois canais
10	2N7000 em TO-92: oito dados e dois controles
10	220 Ω, um em série com cada gate
10	100 kΩ, de cada gate para GND
10	10 kΩ, pull-ups de Q0..Q7, /CE e /OE para VCC da EPROM
8	4,7 kΩ, saídas dos comparadores para 5 V do Mega
1 + 1	22 kΩ e 10 kΩ para gerar VREF; resistores 1%
5	100 nF cerâmicos: VCC, VPP, U1, U2 e VREF; 25 V ou mais
3 linhas	VCC ajustável 5/6 V; VPP ajustável 5/12,5 V; V_AUX fixa 9..15 V
1	Multímetro; osciloscópio recomendado para validar pulsos
Conforme a fonte	Capacitores, limitação de corrente e proteção dos módulos de alimentação

As três linhas podem vir de fontes de bancada ou módulos regulados adequados. Este guia usa as saídas prontas e medidas: não inclui o projeto interno de conversores. Todos os negativos são unidos ao GND do circuito. A linha auxiliar dos comparadores não é a VPP comutada.

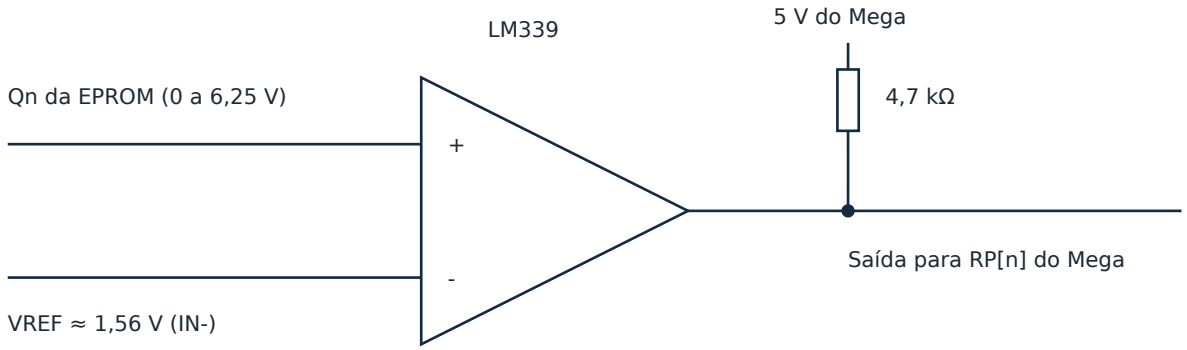
8 • INTERFACE DE DADOS

Escrita por MOSFET e leitura por LM339

Célula de dreno aberto - repetir conforme as tabelas



Repita a célula para Q0 a Q7. O dreno vai ao nó do dado; source ao GND. O gate recebe o pino WP[n] através de 220 Ω e tem pull-down de 100 kΩ. O pull-up de 10 kΩ vai para VCC da EPROM. Antes de habilitar a saída da memória, todos os gates de dados ficam LOW, liberando o barramento.



Alimentação fixa: 9 a 15 V; GND comum

VREF: 5 V do Mega → 22 kΩ → VREF → 10 kΩ → GND, com 100 nF de VREF ao GND. Valor nominal 1,5625 V. Cada saída do LM339 tem seu próprio pull-up de 4,7 kΩ para 5 V do Mega. O nível lógico não é invertido. [2]

Nos 2N7000, identifique G, D e S pelo datasheet do fabricante comprado. Na referência Microchip TO-92: pino 1 = S, 2 = G, 3 = D; a orientação do desenho do encapsulamento é essencial. Não suponha que BS170 ou outro MOSFET tenha a mesma ordem. [3]

8 • INTERFACE DE DADOS / CONTINUAÇÃO

Tabela completa dos oito canais

U1 e U2 são LM339 DIP-14. Em ambos: **pino 3 = V_AUX fixa 9..15 V**, **pino 12 = GND**, e 100 nF entre esses dois pinos. A numeração dos nomes internos de canais varia entre fabricantes; use os números físicos abaixo. [2]

Dado	EPROM	Gate WP	LM339	IN+	IN-	OUT	RP Mega
Q0	11	D2	U1	5	4	2	D47
Q1	12	D3	U1	7	6	1	D38
Q2	13	D4	U1	9	8	14	D37
Q3	15	D5	U1	11	10	13	D40
Q4	16	D6	U2	5	4	2	D41
Q5	17	D7	U2	7	6	1	D42
Q6	18	D8	U2	9	8	14	D43
Q7	19	D9	U2	11	10	13	D44

Todas as entradas IN- da tabela vão a VREF. Todas as IN+ vão ao respectivo nó Qn. Cada OUT vai apenas ao RP indicado e ao seu pull-up para 5 V. **Retire os oito fios diretos EPROM → Mega da montagem A.** Na montagem B, o Mega recebe os dados somente pelas saídas dos comparadores.

Autoteste antes de inserir a EPROM

Com o soquete vazio, VCC e VPP em 5 V, V_AUX ligada e a interface montada, execute:

```
python eprom_pc.py --porta /dev/ttyACM0 autoteste
```

Devem aparecer quatro pares iguais: EXPECT=0 GOT=0, EXPECT=FF GOT=FF, EXPECT=AA GOT=AA e EXPECT=55 GOT=55. O teste confirma o caminho MOSFET → nó → comparador → Mega. Não verifica a EPROM nem a exatidão das fontes. Se falhar, não prossiga para 6 V/12,5 V.

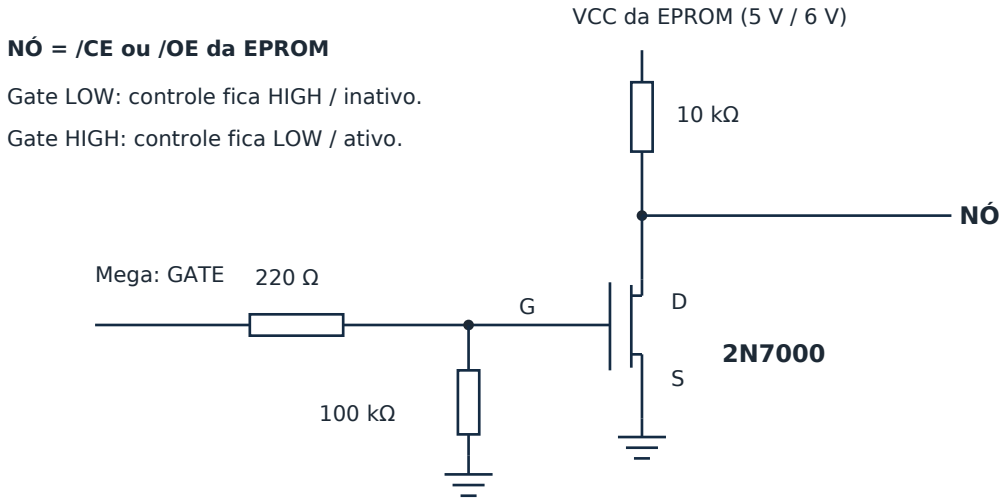
Margens elétricas

Com 10 kΩ para VCC de no máximo 6,25 V, a corrente em um dado LOW é de até 0,625 mA, abaixo dos 2,1 mA usados na especificação do nível baixo da EPROM. A referência próxima de 1,56 V fica entre os níveis TTL de 0 e 1. Com V_AUX de pelo menos 9 V, a faixa de entrada do LM339 cobre o barramento de até 6,25 V. Esta análise não substitui ensaio de montagem e temporização. [1,2]

9 • CONTROLES E ENDEREÇOS

/CE e /OE com estado seguro no reset

Célula de dreno aberto - repetir conforme as tabelas



Gate do Mega	Dreno / nó	Pull-up
D45 através de 220 Ω	Pino 20 da EPROM: /CE	10 kΩ para VCC da EPROM
D46 através de 220 Ω	Pino 22 da EPROM: /OE	10 kΩ para VCC da EPROM

Cada controle usa mais um 2N7000 com source no GND e pull-down de 100 kΩ no gate. Quando o Mega reinicia, os MOSFETs ficam desligados e os pull-ups mantêm /CE e /OE altos. O firmware compensa a inversão. Não mantenha os controles ligados diretamente ao Mega nesta montagem.

Endereços e alimentações

A0..A14 continuam ligados a D22..D36, exatamente na sequência da tabela da montagem A. EPROM pino 14, sources dos MOSFETs, LM339 pino 12, negativos das fontes e GND do Mega ficam no mesmo terra. VCC e VPP passam a vir das fontes externas, **sem ligação ao conector 5V do Mega**. O 5 V do Mega alimenta somente referência e pull-ups das saídas dos comparadores.

Os endereços são diretos. Não deixe o Mega dirigir HIGH quando VCC da EPROM estiver desligada. Na partida, alimente V_AUX e a EPROM em 5 V/5 V antes de energizar o Mega. Ao terminar, use D para deixar endereços em alta impedância antes de retirar VCC. Não desconecte GND durante uma operação.

10 • FONTES MANUAIS

Sequência de bancada para gravar

Etapa	VCC EPROM	VPP EPROM	Situação
Leitura / montagem	5 V	5 V	V_AUX fixa ligada
Preparar programação	Subir para 6 V	Manter 5 V	/CE e /OE HIGH
Programar / verify	6 V	Subir para 12,5 V	Somente após medir as duas
Retornar à leitura	Manter 6 V	Baixar para 5 V	Desarmar antes de ajustar
Verificação final	Baixar para 5 V	5 V	Comparar todos os bytes

Use multímetro no próprio soquete: pino 28 contra 14 e pino 1 contra 14. Ajuste as fontes inicialmente sem o chip e confira novamente sob carga. A especificação de programação é VCC de 5,75 a 6,25 V e VPP de 12,2 a 12,8 V, em ambiente de 20 a 30 °C. Correntes de programação de VCC e VPP são especificadas em até 30 mA cada, além da corrente da interface. Evite que a limitação da fonte derrube as tensões. [1]

Passo a passo

1. Faça o autoteste sem chip; desligue tudo e insira a EPROM apagada.
2. Ligue V_AUX, VCC=5 V e VPP=5 V, depois o Mega. Confira leitura e vazio.
3. Com controles inativos, eleve VCC para 6 V; depois VPP para 12,5 V. Meça.
4. Execute o comando abaixo; confirme GRAVAR apenas com chip e tensões conferidos.
5. O firmware verifica vazio no modo elevado e programa em blocos.
6. Ao pedir, reduza primeiro VPP para 5 V e depois VCC para 5 V. Meça e pressione Enter. O cliente fará três leituras e comparação final exata.

```
python eprom_pc.py --porta /dev/ttyACM0 gravar teste32k.bin
```

Em erro, timeout ou interrupção: o software tenta desarmar, mas não comuta fontes. Confira /CE e /OE altos. Reduza VPP antes de VCC. Se a conexão USB caiu, os pull-downs dos gates ajudam a inibir os controles, mas não dispensam medir as tensões. Não trate DISARMED como fonte desligada.

Para desligar completamente: em 5 V/5 V, envie D; retire VPP, depois VCC, depois V_AUX e USB. Os endereços devem estar em alta impedância antes da retirada de VCC. [1]

11 • ALGORITMO E LIMITES

O que o firmware faz ao programar

Os parâmetros abaixo são específicos do algoritmo antigo ST TS27C256Q. O perfil não é uma seleção universal de “27C256”. [1]

Passo	Comportamento
1. Armar	A exige montagem B e percorre toda a EPROM em verify elevado; só aceita FF em tudo
2. Receber bloco	Até 128 bytes, endereço e CRC32; recusa tamanho, faixa ou CRC inválidos
3. Programar byte	Estabiliza endereço/dados; /OE HIGH; pulso /CE LOW de 1 ms
4. Verificar	Libera todos os MOSFETs de dados; /CE HIGH e /OE LOW; compara o byte
5. Repetir / sobrepulso	Até 25 tentativas, incluindo a 25ª. Ao passar, um sobrepulso de 3 × N ms
6. Finalizar	Desarma; operador retorna a 5 V/5 V; leitura integral e comparação com o BIN

Bits FF não precisam de pulsos; permanecem apagados. A rotina recusa uma transição que exija 0 → 1. O código usa margens de 20 µs nos tempos de setup e hold, superiores aos mínimos de 2 µs; o pulso inicial é de 1 ms, e o maior sobrepulso é de 75 ms. Interrupções ficam suspensas durante o pulso; o programa não depende do relógio do Arduino para medir o tempo total da gravação.

Limites explícitos desta versão

Não há medição automática de tensão, desligamento por hardware, detecção confiável de chip ausente ou acionamento automático das fontes. O blank check pode ser enganado por barramento permanentemente alto. Não há gravação da identificação eletrônica, que usaria tensão elevada em A9; nunca aplique 12 V às linhas de endereço desta montagem.

O leitor foi demonstrado no seu ensaio. O circuito novo com LM339/MOSFETs precisa passar pelo autoteste, leitura estável em 5 V e ensaio de programação em bancada. A compilação e os testes de software não medem ruído, overshoot, fonte sob carga, polaridade real de transistor ou largura física do pulso.

12 • PROTOCOLO E CÓDIGOS

Arquivos e configuração

Arquivo	Uso
codigos/eprom27/eprom27.ino	Firmware único; macro 0 para leitor A ou 1 para gravador B
codigos/eprom_pc.py	Cliente Python 3 com pyserial
codigos/LEIA-ME.txt	Comandos de uso e resumo de montagem

Os apêndices reproduzem integralmente os arquivos. Prefira os arquivos do pacote ao copiar trechos do PDF, para preservar recuos e caracteres. O código é ASCII, exceto uma mensagem de console; os comentários explicam a polaridade do circuito.

```
// Leitor direto:
#define PROGRAMMER_HARDWARE 0

// Gravador com a interface das secoes 7 a 10:
#define PROGRAMMER_HARDWARE 1
```

A macro deve ser escolhida antes de enviar o sketch. Não existe comutação automática entre as duas montagens. Troque também a fiação ao migrar de A para B. O firmware identifica o modo com I e bloqueia gravação na versão READER.

Enquadramento USB

I identifica; T diagnostica; B faz estatísticas; R emite “DATA 32768”, os 32.768 bytes binários e END; S testa a interface B; A arma em verify elevado; D desarma. W recebe endereço de 16 bits little-endian, comprimento de 1 a 128 bytes, os dados e CRC32 little-endian do cabeçalho mais os dados. O computador aguarda OK antes do próximo bloco.

Os dumps são comparados byte a byte em três passagens, além do CRC informativo. A entrada de gravação deve ter exatamente 32.768 bytes. Não há mudança de bancos nem interpretação de cabeçalhos de ROM.

Tempo e memória

As operações percorrem 32 KB com digitalWrite/digitalRead e margens conservadoras; espere dezenas de segundos por ensaio completo. O Mega recebe apenas um bloco por vez. O timeout do cliente é de 60 s por resposta ou trecho recebido; ele não deve ser interpretado como duração máxima do processo inteiro.

APÊNDICE A • FIRMWARE COMPLETO

eprom27.ino

Linhas 1 a 48 de 228

```
001  #include <Arduino.h>
002
003  // 0: leitor direto de 5 V. 1: circuito LM339 + MOSFETs do manual.
004  // NUNCA use 1 na montagem direta: /CE e /OE mudam de polaridade no Mega.
005  #define PROGRAMMER_HARDWARE 0
006
007  const uint8_t AP[15] = {22,23,24,25,26,27,28,29,30,31,32,33,34,35,36};
008  // Mapeamento final padronizado; refaca as ligacoes conforme o manual.
009  const uint8_t RP[8] = {47,38,37,40,41,42,43,44};
010  const uint8_t WP[8] = {2,3,4,5,6,7,8,9};
011  const uint8_t CEP = 45, OEP = 46;
012  const uint32_t SIZE = 32768UL;
013  bool armed = false;
014
015  void ce(bool high) {
016    digitalWrite(CEP, PROGRAMMER_HARDWARE ? !high : high);
017  }
018  void oe(bool high) {
019    digitalWrite(OEP, PROGRAMMER_HARDWARE ? !high : high);
020  }
021  void releaseData() {
022    if (PROGRAMMER_HARDWARE)
023      for (uint8_t i=0; i<8; i++) digitalWrite(WP[i], LOW);
024  }
025  void idleBus() {
026    ce(true); oe(true); releaseData();
027  }
028  void floatAddress() {
029    for (uint8_t i=0; i<15; i++) {
030      pinMode(AP[i], INPUT);
031      digitalWrite(AP[i], LOW); // sem pull-up interno
032    }
033  }
034  void address(uint16_t a) {
035    for (uint8_t i=0; i<15; i++) {
036      digitalWrite(AP[i], (a >> i) & 1);
037      pinMode(AP[i], OUTPUT);
038    }
039  }
040  uint8_t readByte(uint16_t a) {
041    idleBus(); address(a);
042    delayMicroseconds(20);
043    ce(armed); // verify elevado: /CE=HIGH; leitura normal: LOW
044    oe(false);
045    delayMicroseconds(20); // inclui resposta dos comparadores
046    uint8_t v=0;
047    for (uint8_t i=0; i<8; i++)
048      if (digitalRead(RP[i])) v |= uint8_t(1U << i);
```

Listagem de referência. Para copiar ou compilar, use o arquivo original do pacote.

APÊNDICE A • FIRMWARE COMPLETO

eprom27.ino

Linhas 49 a 96 de 228

```
049    idleBus();
050    return v;
051  }
052  uint32_t crcByte(uint32_t c, uint8_t b) {
053    c ^= b;
054    for (uint8_t i=0; i<8; i++)
055      c = (c >> 1) ^ ((c & 1) ? 0xEDB88320UL : 0);
056    return c;
057  }
058  void info() {
059    Serial.println(PROGRAMMER_HARDWARE ?
060      F("EPROM27 V1 PROGRAMMER") : F("EPROM27 V1 READER"));
061  }
062  void statistics() {
063    uint32_t c=0xFFFFFFFFUL, ff=0, zz=0;
064    for (uint32_t a=0; a<SIZE; a++) {
065      uint8_t v=readByte(uint16_t(a));
066      c=crcByte(c,v); ff+=(v==255); zz+=(v==0);
067    }
068    Serial.print(F("CRC32=")); Serial.print(c ^ 0xFFFFFFFFUL,HEX);
069    Serial.print(F(" FF=")); Serial.print(ff);
070    Serial.print(F(" ZERO=")); Serial.print(zz);
071    Serial.println(ff==SIZE ? F(" BLANK") : F(" NOT_BLANK"));
072  }
073  void diagnose() {
074    for (uint8_t i=0; i<3; i++) statistics();
075    uint32_t errors=0; uint8_t mask=0;
076    for (uint32_t a=0; a<SIZE; a++) {
077      uint8_t x=readByte(uint16_t(a));
078      readByte(uint16_t(a ^ 0x7FFFUL));
079      uint8_t y=readByte(uint16_t(a));
080      if (x!=y) {
081        errors++; mask |= (x^y);
082        if (errors<=12) {
083          Serial.print(F("ADDR=")); Serial.print(a,HEX);
084          Serial.print(F(" BEFORE=")); Serial.print(x,HEX);
085          Serial.print(F(" AFTER=")); Serial.println(y,HEX);
086        }
087      }
088    }
089    Serial.print(F("DIFF=")); Serial.print(errors);
090    Serial.print(F(" MASK=")); Serial.println(mask,HEX);
091    Serial.println(F("END"));
092  }
093  void dump() {
094    Serial.println(F("DATA 32768"));
095    uint8_t block[128];
096    for (uint32_t a=0; a<SIZE; a+=sizeof(block)) {
```

Listagem de referência. Para copiar ou compilar, use o arquivo original do pacote.

APÊNDICE A • FIRMWARE COMPLETO

eprom27.ino

Linhas 97 a 144 de 228

```
097      for (uint8_t i=0; i<sizeof(block); i++)
098        block[i]=readByte(uint16_t(a+i));
099      Serial.write(block,sizeof(block));
100    }
101    Serial.println(); Serial.println(F("END"));
102  }
103  bool receive(uint8_t *p, uint16_t n) {
104    uint32_t last=millis(); uint16_t k=0;
105    while (k<n) {
106      if (Serial.available()) { p[k++]=uint8_t(Serial.read()); last=millis(); }
107      else if (uint32_t(millis()-last)>5000UL) return false;
108    }
109    return true;
110  }
111  void driveData(uint8_t v) {
112    for (uint8_t i=0; i<8; i++)
113      digitalWrite(WP[i], (v & (1U << i)) ? LOW : HIGH);
114  }
115  void interfaceTest() {
116    if (!PROGRAMMER_HARDWARE || armed) {
117      Serial.println(F("ERR HARDWARE_OR_ARMED")); return;
118    }
119    idleBus();
120    const uint8_t pattern[4]={0x00,0xFF,0xAA,0x55};
121    for (uint8_t k=0; k<4; k++) {
122      driveData(pattern[k]); delayMicroseconds(50);
123      uint8_t got=0;
124      for (uint8_t i=0; i<8; i++)
125        if (digitalRead(RP[i])) got |= uint8_t(1U << i);
126      Serial.print(F("EXPECT=")); Serial.print(pattern[k],HEX);
127      Serial.print(F(" GOT=")); Serial.println(got,HEX);
128    }
129    idleBus(); Serial.println(F("END"));
130  }
131  void programPulse(uint16_t a, uint8_t v, uint8_t ms) {
132    idleBus(); address(a); driveData(v);
133    delayMicroseconds(20);
134    noInterrupts();
135    ce(false);
136    for (uint8_t i=0; i<ms; i++) delayMicroseconds(1000);
137    ce(true);
138    interrupts();
139    delayMicroseconds(20); // hold dos dados apos /CE subir
140    releaseData(); delayMicroseconds(20);
141  }
142  bool programByte(uint16_t a, uint8_t v) {
143    uint8_t old=readByte(a);
144    if ((old & v)!=v) return false; // exigiria mudar 0 para 1
```

Listagem de referência. Para copiar ou compilar, use o arquivo original do pacote.

APÊNDICE A • FIRMWARE COMPLETO

eprom27.ino

Linhas 145 a 192 de 228

```
145    if (v==0xFF) return old==0xFF;
146    for (uint8_t n=1; n<=25; n++) {
147      programPulse(a,v,1);
148      if (readByte(a)==v) {
149        programPulse(a,v,uint8_t(3*n));
150        return readByte(a)==v;
151      }
152    }
153    return false;
154  }
155  void disarm() {
156    idleBus(); armed=false; floatAddress();
157  }
158  void arm() {
159    if (!PROGRAMMER_HARDWARE) { Serial.println(F("ERR HARDWARE")); return; }
160    // Operador JA ajustou e mediu VCC=6 V e VPP=12.5 V.
161    // Nao ha sensor nem controle automatico de tensao neste projeto.
162    armed=true;
163    for (uint32_t a=0; a<SIZE; a++) {
164      if (readByte(uint16_t(a))!=0xFF) {
165        disarm(); Serial.println(F("ERR NOT_BLANK")); return;
166      }
167    }
168    Serial.println(F("ARMED"));
169  }
170  void writeBlock() {
171    // W + endereco LE16 + tamanho U8 (1..128) + dados + CRC32 LE32.
172    uint8_t h[3], data[128], tail[4];
173    if (!receive(h,3)) { disarm(); Serial.println(F("ERR TIMEOUT")); return; }
174    uint16_t a=uint16_t(h[0]) | (uint16_t(h[1])<<8);
175    uint8_t n=h[2];
176    if (!n || n>128 || uint32_t(a)+n>SIZE) {
177      disarm(); Serial.println(F("ERR RANGE")); return;
178    }
179    if (!receive(data,n) || !receive(tail,4)) {
180      disarm(); Serial.println(F("ERR TIMEOUT")); return;
181    }
182    uint32_t c=0xFFFFFFFFUL;
183    for (uint8_t i=0; i<3; i++) c=crcByte(c,h[i]);
184    for (uint8_t i=0; i<n; i++) c=crcByte(c,data[i]);
185    c ^= 0xFFFFFFFFUL;
186    uint32_t given=0;
187    for (uint8_t i=0; i<4; i++) given |= uint32_t(tail[i]) << (8*i);
188    if (c!=given) { disarm(); Serial.println(F("ERR CRC")); return; }
189    if (!armed || !PROGRAMMER_HARDWARE) {
190      Serial.println(F("ERR NOT_ARMED")); return;
191    }
192    for (uint8_t i=0; i<n; i++) {
```

Listagem de referência. Para copiar ou compilar, use o arquivo original do pacote.

APÊNDICE A • FIRMWARE COMPLETO

eprom27.ino

Linhas 193 a 228 de 228

```
193      if (!programByte(uint16_t(a+i),data[i])) {
194        disarm(); Serial.print(F("ERR VERIFY "));
195        Serial.println(uint16_t(a+i),HEX); return;
196      }
197    }
198    Serial.println(F("OK"));
199  }
200  void setup() {
201    digitalWrite(CEP, PROGRAMMER_HARDWARE ? LOW : HIGH);
202    digitalWrite(OEP, PROGRAMMER_HARDWARE ? LOW : HIGH);
203    pinMode(CEP,OUTPUT); pinMode(OEP,OUTPUT);
204    for (uint8_t i=0; i<8; i++) {
205      pinMode(RP[i],INPUT);
206      if (PROGRAMMER_HARDWARE) {
207        digitalWrite(WP[i],LOW); pinMode(WP[i],OUTPUT);
208      }
209    }
210    floatAddress(); idleBus();
211    Serial.begin(115200); Serial.println(F("READY"));
212  }
213  void loop() {
214    if (!Serial.available()) return;
215    char cmd=char(Serial.read());
216    if (cmd=='I') info();
217    else if (cmd=='S') interfaceTest();
218    else if (cmd=='D') { disarm(); Serial.println(F("DISARMED")); }
219    else if (cmd=='A') arm();
220    else if (cmd=='W') writeBlock();
221    else if (cmd=='B' || cmd=='T' || cmd=='R') {
222      if (armed) { Serial.println(F("ERR ARMED")); return; }
223      if (cmd=='B') { statistics(); Serial.println(F("END")); }
224      if (cmd=='T') diagnose();
225      if (cmd=='R') dump();
226      idleBus(); floatAddress();
227    }
228  }
```

Listagem de referência. Para copiar ou compilar, use o arquivo original do pacote.

APÊNDICE B • CLIENTE PYTHON COMPLETO

eprom_pc.py

Linhas 1 a 48 de 166

```
001  #!/usr/bin/env python3
002  """Cliente USB para eprom27.ino. Requer pyserial; nenhum pacote de ROMs."""
003  import argparse
004  from pathlib import Path
005  import struct
006  import time
007  import zlib
008  import serial
009
010  SIZE = 32768
011
012  def exact(s, n):
013      data = bytearray()
014      while len(data) < n:
015          part = s.read(n - len(data))
016          if not part:
017              raise RuntimeError('Timeout: cabo, firmware ou alimentacao.')
018          data.extend(part)
019      return bytes(data)
020
021  def line(s):
022      value = s.readline()
023      if not value.endswith(b'\n'):
024          raise RuntimeError('Resposta incompleta ou timeout.')
025      text = value.decode('ascii', errors='replace').strip()
026      if text.startswith('ERR'):
027          raise RuntimeError(text)
028      return text
029
030  def identify(s):
031      time.sleep(2.5)  # abrir a serial costuma reiniciar o Mega
032      s.reset_input_buffer()
033      s.write(b'I')
034      result = line(s)
035      if not result.startswith('EPROM27 V1 '):
036          raise RuntimeError('Firmware inesperado: ' + result)
037      print(result)
038      return result
039
040  def read_image(s):
041      s.write(b'R')
042      if line(s) != 'DATA 32768':
043          raise RuntimeError('Cabecalho de leitura inesperado.')
044      data = exact(s, SIZE)
045      if exact(s, 2) != b'\r\n' or line(s) != 'END':
046          raise RuntimeError('Final de leitura inesperado.')
047      return data
048
```

Listagem de referência. Para copiar ou compilar, use o arquivo original do pacote.

APÊNDICE B • CLIENTE PYTHON COMPLETO

eprom_pc.py

Linhas 49 a 96 de 166

```
049  def stable_read(s):
050      images = []
051      for n in range(3):
052          image = read_image(s)
053          print(f'Leitura {n+1}: CRC32={zlib.crc32(image):08X} '
054                f'FF={image.count(255)} ZERO={image.count(0)}')
055          if images and image != images[0]:
056              bad = next(i for i in range(SIZE) if image[i] != images[0][i])
057              raise RuntimeError(f'Leituras diferentes em 0x{bad:04X}; nao salvar.')
058          images.append(image)
059      return images[0]
060
061  def input_image(path):
062      data = Path(path).read_bytes()
063      if len(data) != SIZE:
064          raise RuntimeError('Use imagem de exatamente 32768 bytes. '
065                             'Nao ha padding nem reorganizacao automatica.')
066      return data
067
068  def text_command(s, command):
069      s.write(command)
070      while True:
071          result = line(s)
072          if result == 'END':
073              break
074          print(result)
075
076  def safe_disarm(s):
077      try:
078          s.write(b'D')
079          result = line(s)
080          print(result)
081          return result == 'DISARMED'
082      except Exception:
083          print('Sem confirmacao de D: confira /CE e /OE no circuito.')
084          return False
085
086  def write_image(s, data, identity):
087      if not identity.endswith('PROGRAMMER'):
088          raise RuntimeError('Gravacao bloqueada no firmware READER.')
089      print('Circuito LM339 + MOSFETs do manual obrigatorio.')
090      print('Meça VCC=6.0 V e VPP=12.5 V; /CE e /OE devem estar altos.')
091      disarmed = False
092      try:
093          if input('Digite GRAVAR para confirmar as fontes e o chip: ') != 'GRAVAR':
094              raise RuntimeError('Operacao cancelada antes de armar.')
095          print('Conferindo EPROM vazia no modo de verify elevado...')
096          s.write(b'A')
```

Listagem de referência. Para copiar ou compilar, use o arquivo original do pacote.

APÊNDICE B • CLIENTE PYTHON COMPLETO

eprom_pc.py

Linhas 97 a 144 de 166

```
097          if line(s) != 'ARMED':
098              raise RuntimeError('Nao foi possivel armar.')
099          for addr in range(0, SIZE, 128):
100              block = data[addr:addr+128]
101              packet = struct.pack('<HB', addr, len(block)) + block
102              s.write(b'W' + packet + struct.pack('<I', zlib.crc32(packet)))
103              if line(s) != 'OK':
104                  raise RuntimeError(f'Falha no bloco 0x{addr:04X}.')
105              print(f'Verificado em tensao de programacao: {addr+128}/{SIZE}')
106      finally:
107          disarmed = safe_disarm(s)
108          print('O comando D NAO altera fontes externas.')
109          print('Baixe PRIMEIRO VPP para 5 V; DEPOIS VCC para 5 V.')
110      if not disarmed:
111          raise RuntimeError('Nao foi confirmado o estado seguro dos controles.')
112      input('Meça ambas. Pressione Enter somente apos confirmar 5 V: ')
113      actual = stable_read(s)
114      if actual != data:
115          bad = next(i for i in range(SIZE) if actual[i] != data[i])
116          raise RuntimeError(f'Verificacao final falhou em 0x{bad:04X}.')
117      print('GRAVACAO E VERIFICACAO FINAL OK.')
118
119  def main():
120      p = argparse.ArgumentParser(description=__doc__)
121      p.add_argument('--porta', required=True, help='/dev/ttyACM0 ou COM3')
122      p.add_argument('acao', choices=['info','teste','autoteste','vazio','ler','verificar','gravar'])
123      p.add_argument('arquivo', nargs='?')
124      args = p.parse_args()
125      if args.acao in ('ler','verificar','gravar') and not args.arquivo:
126          p.error('Esta acao exige arquivo.')
127      if args.acao == 'ler' and Path(args.arquivo).exists():
128          p.error('Arquivo ja existe. Escolha outro nome para preservar a copia.')
129      expected = input_image(args.arquivo) if args.acao in ('verificar','gravar') else None
130      with serial.Serial(args.porta,115200,timeout=60,write_timeout=10) as s:
131          try:
132              identity = identify(s)
133              if args.acao == 'teste':
134                  text_command(s,b'T')
135              elif args.acao == 'autoteste':
136                  text_command(s,b'S')
137              elif args.acao == 'vazio':
138                  data = stable_read(s)
139                  print('VAZIA: todos os bytes FF.' if data == b'\xff'*SIZE
140                        else 'NAO VAZIA: existem bytes diferentes de FF.')
141              elif args.acao == 'ler':
142                  data = stable_read(s)
143                  with open(args.arquivo,'xb') as out:
144                      out.write(data)
```

Listagem de referência. Para copiar ou compilar, use o arquivo original do pacote.

APÊNDICE B • CLIENTE PYTHON COMPLETO

eprom_pc.py

Linhas 145 a 166 de 166

```
145                 print('Salvos 32768 bytes em', args.arquivo)
146             elif args.acao == 'verificar':
147                 data = stable_read(s)
148                 if data != expected:
149                     bad = next(i for i in range(SIZE) if data[i] != expected[i])
150                     raise RuntimeError(f'Conteudo diferente em 0x{bad:04X}.')
151                 print('Arquivo e EPROM identicos byte a byte.')
152             elif args.acao == 'gravar':
153                 write_image(s,expected,identity)
154         finally:
155             # Desabilita controles e deixa enderecos em alta impedancia.
156             safe_disarm(s)
157             if args.acao == 'gravar':
158                 print('Se ainda elevadas: baixe VPP para 5 V antes de VCC.')
159
160 if __name__ == '__main__':
161     try:
162         main()
163     except (RuntimeError,OSError,serial.SerialException) as exc:
164         raise SystemExit('ERRO: ' + str(exc))
165     except KeyboardInterrupt:
166         raise SystemExit('Interrompido. Se fontes elevadas, baixe VPP antes de VCC.')
```

Listagem de referência. Para copiar ou compilar, use o arquivo original do pacote.

REFERÊNCIA RÁPIDA

Checklist e fontes técnicas

Antes de ler	Antes de gravar
Montagem A em 5 V/5 V ou B no modo de leitura	Família TS27C256Q e encapsulamento confirmados
Mapa AP/RP igual à fiação	Montagem B completa; macro = 1
Capacitores próximos e fios curtos	Autoteste 00/FF/AA/55 sem falhas
Três dumps idênticos; DIFF=0	Apagamento UV e blank check
BIN conhecido para afirmar conteúdo correto	VCC e VPP medidas; ordem das fontes respeitada

Validação dos arquivos deste manual

Compilação real com Arduino CLI e core arduino:avr 1.8.8, alvo arduino:avr:mega: **leitor 4.860 bytes de flash / 211 bytes globais**; **gravador 5.678 bytes de flash / 223 bytes globais**. Cliente Python passou em compilação de sintaxe. Revisão independente verificou algoritmo, polaridades, dump e tratamento de erros. Os resultados adicionais de simulação acompanham o pacote em VALIDACAO.txt.

A compilação não valida a fiação. A parte de programação continua sem ensaio físico. Confirme a marcação completa e valide a montagem B antes de inserir a EPROM ou elevar tensões.

Fontes primárias

[1] SGS-Thomson, Memory Products Databook, junho/1988. TS27C256Q, páginas impressas 121 a 129; parâmetros e algoritmo, 126 a 128. Abrir documento

[2] Texas Instruments, LM339, datasheet SLCS006Z. Pinagem DIP-14, modo comum e saída coletor aberto. Abrir documento

[3] Microchip, 2N7000, DS20005695A. Terminais, encapsulamento e características elétricas. Abrir documento

[4] Arduino, Mega 2560 Rev3. Placa, pinagem e especificações. Abrir documento

[5] Microchip, ATmega2560. Limites elétricos do microcontrolador. Abrir documento

Figura de pinagem: imagem enviada pelo usuário nesta conversa. Diagramas de interface, código e texto deste manual: elaboração original para esta montagem. Consulta e revisão: 24/10/2026.